

AI-Argus-Harness

Structure & Design

Enterprise AI security harness - evidence-first, identity-aware

Author: Sam Gupta

Core principle

AI is an evidence INTERPRETER, never an evidence CREATOR.

Every finding must carry asset + evidence + identity path + impact + confidence + remediation + owner before it is promoted to enterprise reporting; otherwise it is routed to human review.

Pipeline phases (S0 -> S10)

 Discovery	2 stage(s): S0, S1
 Identity	1 stage(s): S1.5
 Threat Analysis	3 stage(s): S2, S3, S4
 Evidence & Verification	2 stage(s): S5, S5.5
 Reliability Review	2 stage(s): S6, S6.5
 Risk Intelligence	3 stage(s): S7, S8, S8.5
 Reporting & Governance	2 stage(s): S9, S10

Pipeline: **Discovery -> Identity -> Verification -> Risk Intelligence -> Reporting**

Deterministic scanners produce evidence; the reliability layer gates it; scoring & chaining add risk intelligence.

Version 0.1.0

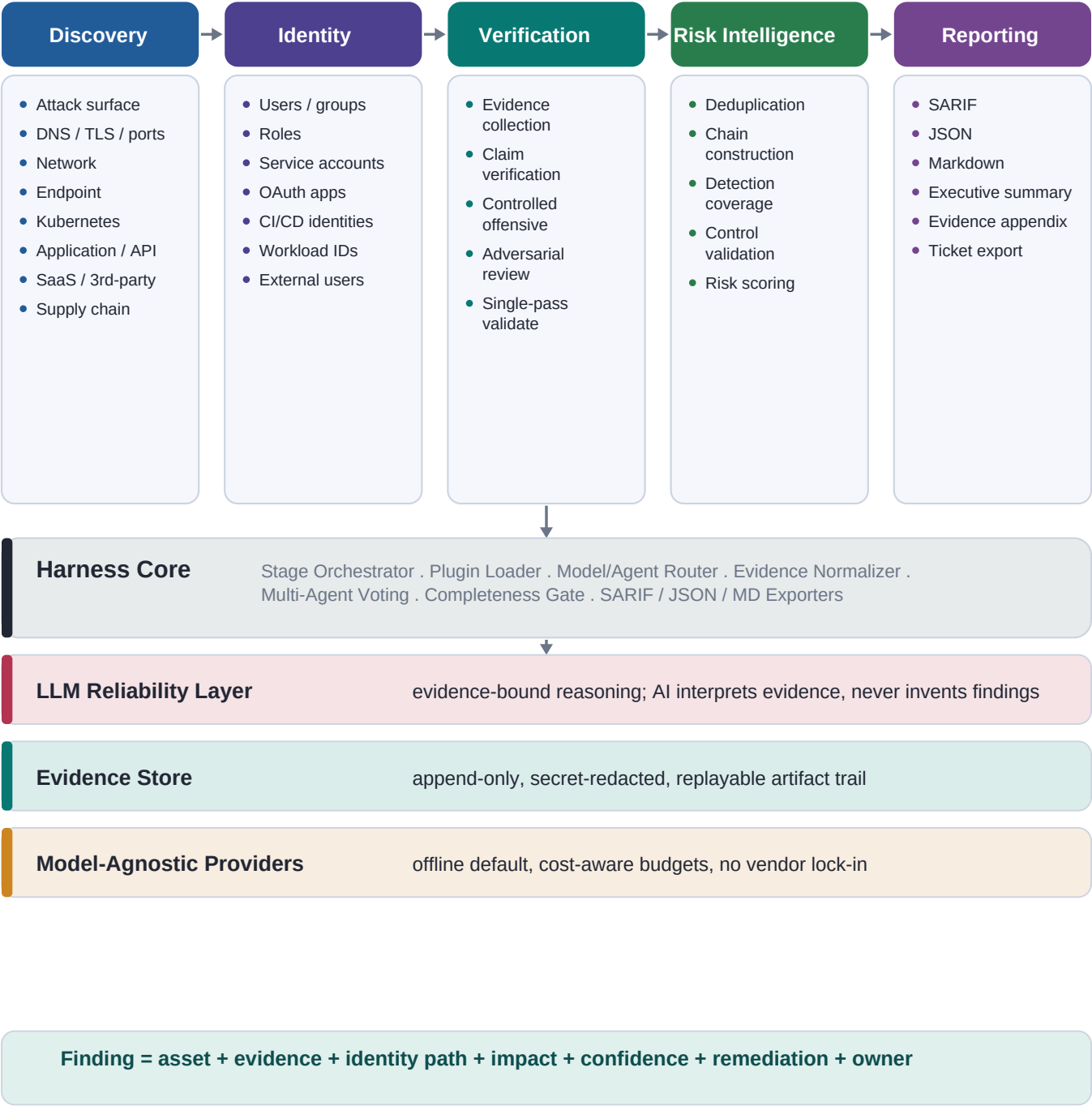
Author: Sam Gupta

Generated 2026-06-25

Document: structure, design and the key feature at each stage.

1. Structure & Design

Five pillars around a Harness Core, with cross-cutting reliability, evidence and model-provider layers.



2. Enterprise Stage Pipeline

Each phase groups stages; every stage card highlights its key FEATURE.



3. Overview

AI-Argus-Harness maps, analyzes, verifies and reports security exposure across modern enterprise environments: external attack surface, cloud, identity, Kubernetes, applications/APIs, SaaS and the software supply chain. It is evidence-first: deterministic scanners produce structured evidence and AI only reasons over that evidence, which sharply reduces hallucinations and keeps runs reproducible and audit-ready.

Core principle

AI is an evidence interpreter, never an evidence creator. A finding is only promoted to enterprise reporting when it carries: asset, evidence, identity path (for high/critical), impact, confidence, remediation and owner. Anything incomplete is routed to human review (or dropped in strict mode).

4. Design Principles

- Lightweight and cross-platform; the core harness has zero hard dependencies.
 - Model-agnostic with no vendor lock-in; offline provider is the default.
 - Offline-capable scanning; the tool always runs even with no network or API key.
 - Deterministic and reproducible scoring and severity.
 - Cost-aware with per-day, per-month and per-scan budgets.
 - Plugin-driven scanners discoverable via Python entry points.
 - Read-only and safe by default; active checks require explicit safe/auth modes.
 - Enterprise-ready: secret redaction, replayable evidence, SARIF and ticket export.
-

5. Module Map

- core/ - stage orchestrator, plugin loader, run context, stage model, credential preflight (authenticated/non-authenticated scanning).
 - graph/ - enterprise asset graph and first-class identity/access graph.
 - evidence/ - append-only, redacted, replayable evidence store and normalizer.
 - reliability/ - claim verifier, adversarial reviewer, validator, completeness gate.
 - scoring/ - deterministic additive risk model and deduplication.
 - chaining/ - attack-path chain construction.
 - reporting/ - JSON, SARIF, Markdown, executive summary and ticket exporters.
 - scanners/ - 11 deterministic scanners (domain, network, secrets, supply_chain, aws, gcp, kubernetes, saas, cicd, data_exposure, application).
 - llm/ - model-agnostic provider abstraction with cost tracking (offline default).
-

6. Stage-by-Stage Feature Highlights

S0 - Credential Preflight

Feature: At the start of a scan, chooses an authenticated or non-authenticated run. For an authenticated scan it discovers credentials for the services about to be scanned (operator override, inventory credential census, environment variables, standard credential files); if none are found it prompts for a key or continues non-authenticated. Records only references (env-var name / file path), never secret values, and probes no third-party hosts. This auth context applies to stages S1-S5.5.

- Produces: Per-service credential status (authenticated / unauthenticated)
- Module: core/auth.py, core/orchestrator.py (S0)

S1 - Attack-Surface Mapper

Feature: Maps the external footprint (DNS, TLS, open ports, technologies, sensitive subdomains) and builds the enterprise asset graph.

- Produces: Assets + attack-surface evidence
- Module: scanners/domain.py, scanners/network.py, graph/asset_graph.py

S1.5 - Identity & Access Graph Builder

Feature: Treats identity as a first-class graph; ingests users, roles, service accounts, OAuth apps and CI identities; finds privileged and blind-spot paths.

- Produces: Identity graph + access edges
- Module: graph/identity_graph.py

S2 - Threat Modeler

Feature: Derives the likely threats against each discovered asset to focus research.

- Produces: Threat hypotheses per asset
- Module: core/orchestrator.py (S2)

S3 - Research Strategist

Feature: Plans which research lenses and scanners to run for the target and budget.

- Produces: Research plan
- Module: core/orchestrator.py (S3)

S4 - Research Lenses

Feature: Runs deterministic, plugin-based scanners in parallel to gather structured evidence (cloud, k8s, secrets, supply chain, SaaS, application).

- Produces: Candidate findings + evidence
- Module: scanners/*

S5 - Evidence Collector

Feature: Normalizes and de-duplicates source-bound evidence from all scanners; redacts secrets; persists evidence for replay.

- Produces: Normalized evidence store
- Module: evidence/normalizer.py, evidence/store.py

S5.5 - Controlled Offensive Verification

Feature: Safely verifies findings with no destructive actions; honors passive/safe/auth/stealth modes; no exploitation, persistence or exfiltration.

- Produces: Verified vs rejected claims
- Module: reliability/claim_verifier.py

S6 - Adversarial Reviewer

Feature: Challenges weak or unsupported conclusions and tracks multi-agent disagreement; can add an independent, evidence-bound model opinion.

- Produces: Challenged / adjusted findings
- Module: reliability/adversarial_reviewer.py

S6.5 - Single-Pass Validator

Feature: Performs structural and schema validation of every finding's shape and fields.

- Produces: Validated findings
- Module: reliability/validator.py

S7 - Deduplication

Feature: Collapses related findings by asset, root cause, identity path and remediation owner.

- Produces: Unique findings
- Module: scoring/dedup.py

S8 - Chain Construction

Feature: Links findings that share assets or identities into multi-step attack paths.

- Produces: Attack-path chains
- Module: chaining/chain_constructor.py

S8.5 - Detection & Control Coverage

Feature: Reviews detection blind spots and weak controls, feeding the risk score.

- Produces: Detection-gap signals
- Module: scoring/risk_scoring.py

S9 - Report Generator

Feature: Deterministically scores and prioritizes findings, then emits SARIF, JSON, Markdown, executive summary and tickets.

- Produces: Enterprise reports
- Module: scoring/risk_scoring.py, reporting/*

S10 - Human Review / Exceptions

Feature: The completeness gate promotes only complete findings; the rest are routed to a human review / exception workflow.

- Produces: Review queue + promoted findings
- Module: reliability/completeness_gate.py

7. Authenticated vs Non-Authenticated Scanning

At the start of every scan the operator chooses an authenticated or a non-authenticated run. A non-authenticated scan requires no key and behaves as before; an authenticated scan adds the S0 credential preflight, whose auth context applies to stages S1, S1.5, S2, S3, S4, S5 and S5.5.

Credential discovery (local, read-only)

Credentials are resolved in priority order: explicit operator overrides, the inventory credential census, standard environment variables, then standard credential files (for example `~/.aws/credentials`, `~/.kube/config`, gcloud ADC). Only references - an environment-variable name or a file path - and a presence flag are recorded; secret values are never read, printed, transmitted or stored, and no third-party hosts are probed to harvest keys.

Fallback when no key is found

- Prompt for a key per missing service (an env-var name or an `@file` path), or

- Continue as a non-authenticated scan for the services without a credential.

Scope & visibility

- Credentialed services: aws, gcp, kubernetes, saas, cicd; others run external / optional.
- Per-service status appears in the scan summary, the JSON report (auth block) and the evidence trail.
- Targets may be a single target, multiple targets, or --all-targets (one combined inventory run).

8. Reliability Rules (Hallucination Reduction)

- No evidence, no finding.
- No source, no finding (evidence must be source-bound).
- No identity path, no high/critical severity.
- No impact, no severity.
- No owner, no promotion to enterprise reporting.
- No remediation, no ticket.
- Below the confidence threshold in strict mode, route to human review or drop.
- Adversarial review challenges weak claims and records multi-agent disagreement.

9. Risk Scoring & Severity

Risk is a deterministic additive model: Exposure + Privilege + Exploitability + IdentityPathStrength + DataSensitivity + BusinessCriticality + Chainability + DetectionGap + ControlWeakness + Confidence - CompensatingControls.

Severity is derived from the score plus qualitative signals: internet exposure with a privileged identity path to sensitive data and weak detection escalates to critical.

10. Coverage (Scanners)

- Live, read-only, stealth-aware: domain (DNS/TLS/subdomains), network (ports).
- Offline code-path: secrets, supply_chain (SBOM + typosquat), cicd pipelines.
- Inventory-snapshot, credentialed-equivalent: aws, gcp, kubernetes, saas, data_exposure, application - deterministic policy checks, fully replayable.

11. CLI Quick Reference

- argus init - first-time setup (provider, model, budget).
- argus scan <target> - run the stage pipeline (domain, CIDR, path, or scanner).
- argus quick <target> - fast minimal-cost attack-surface scan.
- argus scan --stage S1,S2,S4 / --until S6 - run a subset of stages.
- Cost modes: --minimal | --balanced | --research | --deep-research | --offline.
- Stealth modes: --passive | --safe | --auth | --stealth; --strict for gating.
- Authenticated scan: --auth-scan / --no-auth-scan (distinct from the --auth stealth mode); --auth-key SERVICE=ENV_VAR or SERVICE=@/path supplies a reference, never a secret.
- Targets: argus scan host-a host-b ... or --all-targets (one combined inventory run).
- Incremental & governance: --diff (new/fixed/unchanged vs baseline); argus suppress / suppressions for accepted risks.
- argus graph | identity | explain <id> | replay <id> | evidence | cost | doctor.

12. Reports & Outputs

- report.json - full machine-readable run (findings, graphs, evidence, chains).
- report.sarif - SARIF 2.1.0 for code-scanning dashboards.
- report.md - engineer-facing detail with evidence and identity paths.
- executive.md - leadership-ready risk summary.
- tickets.json - owner-based export for Jira / ServiceNow.

13. Enterprise Readiness

- Security: read-only by default, secret redaction, no exploitation or exfiltration.
- Reliability: completeness gate, adversarial review, deterministic reproducibility.
- Scale: parallel scanners, incremental scans, central inventory snapshots.
- Governance: budgets, cost dashboard, audit-ready replayable evidence.